

La traduction

Un système de traduction est mis en place dans le projet, J'ai utilisé le système de localisation fourni par Laravel. Leur fonctionnement est assez simple. Voici comment mettre le en place.

Le dossier Lang

Il existe un dossier **lang** qui contient les fichiers de traduction. Par défaut Laravel n'affiche pas ce dossier pour cela vous devez faire la commande :

```
`php artisan lang:publish`
```

Dans ce dossier **lang** vous trouverez les sous-dossiers pour chaque langue prise en charge par votre application. Attention à bien nommer les sous-dossiers des langues conformément à la [norme ISO](#).

Le fichier de traduction

Le fichier de traduction peut se faire de deux façons soit dansg un fichier php ou en json. Ces fichiers renvoient ensuite un tableau de chaînes de caractères indexées. Voici un exemple pour le php (celui que j'ai pris pour le projet) :

```
<?php
// lang/en/Home.php
return [
    'welcome_home' => 'Welcome to our application!',
];
```

Dans les commentaires le fichier est dans la langue anglaise (en), si maintenant je voudrais ajouter le français alors je créer un dossier fr dans lang et je copie le code ci-dessus et change welcome to our application en français.

Si vous souhaitez utiliser la version **json** rendez-vous sur [ce lien](#). Je vous conseil de nommer les fichiers de traduction par le nom de la page concerné cela facilite la navigation des fichiers.

La récupération de la traduction

Maintenant, pour utiliser les traductions mises en place sur nos pages blade on utiliseras le formatage suivant : `{{ __(home.welcome_home) }}`. À savoir que laravel parcourt le dossier lang pour aller chercher le mot clef dans la langue actuellement en charge.

Déterminer la langue actuelle

Dans **config/app.php** vous avez deux éléments utiles ;

- **APP_LOCALE** : Vous définissez la langue par défaut à utiliser.
- **APP_FALLBACK_LOCALE** : Si la langue par défaut est pas disponible on utilisera celle-ci.

```
'locale' => env('APP_LOCALE', 'en'),  
'fallback_locale' => env('APP_FALLBACK_LOCALE', 'fr'),
```

Changement de la langue par l'utilisateur

Pour réaliser ceci, j'ai mis en place deux liens hypertexte permettant de basculer entre le Français et l'Anglais. Lorsqu'un utilisateur clique sur une langue, la requête est interceptée par une route spécifique dans le fichier web.php.

- **Action** : Génère une URL via la route nommée lang.switch en passant le code pays (fr ou en) comme paramètre.
- **Validation** : Par mesure de sécurité, on vérifie que la langue reçue en paramètre appartient à une liste autorisée (un tableau contenant 'fr' et 'en').
- **Mise en mémoire** : Si la langue est valide, elle est stockée dans la **session** de l'utilisateur sous la clé locale. Cela permet de conserver le choix de l'utilisateur durant toute sa navigation.

```
Route::get('lang/{locale}', function ($locale) {  
    if (in_array($locale, ['en', 'fr'])) {  
        session()->put('locale', $locale);  
    }  
    return redirect()->back();  
})->name('lang.switch');
```

```
<div class="lang-selector">  
    <a href="{{ route('lang.switch', 'fr') }}"  
        class="lang-link {{ app()->getLocale() == 'fr' ? 'active' : '' }}">FR</a>  
    <span style="color: #e5e7eb;">|</span>  
    <a href="{{ route('lang.switch', 'en') }}"  
        class="lang-link {{ app()->getLocale() == 'en' ? 'active' : '' }}">EN</a>  
</div>
```

La vérification par mail

Un système de vérification est mis en place dans le projet, Laravel fourni aussi ce service alors je l'ai également utilisé. Voici son fonctionnement.

Préparer le model

Dans **App/Models/User.php** vérifier que le model des utilisateurs implémente bien le MustVerifyEmail ;

```
<?php
namespace App\Models;

use Illuminate\Contracts\Auth\MustVerifyEmail;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;

class User extends Authenticatable implements MustVerifyEmail
{
    use Notifiable;

    // ...
}
```

Préparation de la base de données

Ensuite, votre table des utilisateurs doit contenir une colonne **email_verified_at** pour enregistrer la date et l'heure de vérification de l'adresse e-mail de l'utilisateur. Par défaut, cette colonne est incluse dans :

- **database/migrations/0001_01_01_000000_create_users_table.php**

```
Schema::create('users', function (Blueprint $table) {
    $table->id('id_user');
    $table->string('name');
    $table->string('email')->unique();
    $table->timestamp('email_verified_at')->nullable();
    $table->string('password');
    $table->rememberToken();
    $table->timestamps();
    $table->string('role')->default('user');
});
```

Le routage

Cette vérification par mail repose sur trois routes. Premièrement, une route nommée **verification.notice** sera nécessaire pour afficher la vue qui invitera l'utilisateur à cliquer sur le lien de vérification d'adresse mail.

Deuxièmement, une route nommée **verification.verify** sera nécessaire pour gérer les requêtes générées lorsque l'utilisateur clique sur le lien de vérification qu'il aura reçu par e-mail. C'est aussi ici qu'on redirigera l'utilisateur sur la page de notre choix après la vérification.

Et troisièmement, une procédure appelée **verification.send** sera nécessaire pour renvoyer un lien de vérification si l'utilisateur perd accidentellement le premier lien de vérification. Voyons le code de ces routes ;

```
Route::middleware('auth')->group(function () {
    Route::get('verify-email', EmailVerificationPromptController::class)
        ->name('verification.notice');

    Route::get('verify-email/{id}/{hash}', VerifyEmailController::class)
        ->middleware(['signed', 'throttle:6,1'])
        ->name('verification.verify');

    Route::post('email/verification-notification', [EmailVerificationNotificationController::class, 'store'])
        ->middleware('throttle:6,1')
        ->name('verification.send');
```

Chacune de ces routes ont un Controller bien distinct qui se charge d'effectuer leurs requêtes et le travail pour lesquelles elles sont missionnées.

La protection des routes

Évidemment que mettre la vérification par e-mail en place c'est top ! Mais ce qui n'est pas cool revanche c'est de la contourner en changeant volontairement de page par une url style : jobDispatcher/home. C'est pourquoi j'ai dû mettre en place dans les routes un élément qui s'assure que l'utilisateur connecté possède bien une adresse mail vérifiée !

Cet élément est un alias inclut par Laravel c'est le **verified** il nous suffit de l'accrocher à un middleware et c'est tout !

```
Route::get('/home', [JobController::class, 'index'])
    ->middleware(['auth', 'verified'])
    ->name('home');
```

Personnalisation du mail

Par défaut, Laravel fournit un mail déjà construit et utilisable, en cas de besoins si vous souhaitez modifier vous avez deux possibilités depuis dans la méthode boot du **AppServiceProvider** ou vous pourrez modifier les éléments essentiels comme le sujet du mail, le corps du mail et le texte du bouton de vérification. Si cela vous suffit pas vous pouvez compléter la modification en ayant dans **ressources/views/vendor** vous trouverez deux dossiers mail et notifications ou vous pouvez faire vos modifications parfaite mais attention laravel ne fournit pas ce dossier vendor par défaut vous devez taper les commandes suivantes :

- ``php artisan vendor :publish --tag=laravel-mail``
- ``php artisan vendor :publish --tag=laravel-notifications``

Docker Compose

J'ai créé un fichier docker compose afin de permettre de travailler sur son propre environnement local. Ce choix s'explique par les problèmes rencontrés au fur et à mesure du développement.

Par exemple durant le projet nous avons tous utilisé la même base de données en même temps via une seule adresse IP et parfois quand une personne fait des travaux de modification directement sur la base de données et bien tout le monde hérite des modifications en direct et donc par la même occasion des erreurs.

Du coup grâce au docker compose nous pouvons directement passer en mode local sans dépendre de quelqu'un.

```
1  services :
2    >Run Service
3    db : # Service de base de données MySQL
4      image : mysql:8.0
5      restart : always
6      container_name : MySQL
7      ports :
8        - "3306:3306"
9      environment :
10       - MYSQL_ROOT_PASSWORD=dispatcherRoot1234
11       - MYSQL_DATABASE=printer_db
12     volumes :
13       - db_data:/var/lib/mysql
14
15   >Run Service
16   phpmyadmin: # Service phpMyAdmin pour gérer la base de données MySQL
17     image: phpmyadmin
18     restart: always
19     ports:
20       - 8080:80
21     environment:
22       - PMA_ARBITRARY=1
23
24   >Run Service
25   portainerServer: # Service Portainer pour la gestion de Docker
26     image: portainer/portainer-ce:latest
27     container_name: portainer_server
28     restart: always
29     ports:
30       - "9000:9000"
31     volumes:
32       - /var/run/docker.sock:/var/run/docker.sock
33       - portainer_data:/data
34
35   volumes: # Volumes pour persister les données de MySQL et Portainer
36     db_data:
37     portainer_data:
```

Viewer 3D

Voici le plus travail le plus gros que j'ai dû faire sur ce projet. Il fallait pouvoir gérer l'orientation d'une pièce 3D sur la face du plateau d'impression. Pour se faire j'ai utilisé la librairie three.js que Monsieur Jean-Christophe Mathez m'a présenté.

La documentation de cette librairie se trouve sur ce [lien](#).

Un spike

Tout d'abord comme cette librairie à mes yeux est considérée comme une dette technique. Je me suis mis à faire un spike afin d'en apprendre plus sur comment l'utiliser.

Finalement j'ai vu que la librairie est composée d'énormément de méthodes et de fonctionnalités disponibles et le point vraiment fort c'est que nous ne sommes pas obligés de tous les prendre. L'objectif de cette partie étant de gérer l'orientation de la pièce sur son plateau d'impression j'ai donc choisi de faire et utiliser les fonctionnalités suivantes ;

- Le chargement d'un fichier STL.
- La sélection d'une face de la pièce.
- L'orientation de la face sélectionnée vers le plateau d'impression.
- Reset de l'orientation de la pièce.
-

L'interface Utilisateur

J'ai commencé par réaliser l'interface web l'objectif était d'avoir deux pages une pour la création d'un job d'impression et l'autre pour la modification. J'ai donc mis dans le code html et à l'intérieur des balises (boutons, canvas, input) des id.

Ceci pour la simple raison que les requêtes que je fais pour valider la création ou modification se fera en javascript avec l'appel de fetch. Ce choix s'explique par le fait que j'utilise le code javascript pour gérer la librairie et donc sauvegarder le nouveau fichier **.stl** générer par ce même code javascript.

Importer la librairie

J'ai fait deux imports distinct le premier import se charge de récupérer la librairie three et le deuxième import se charge d'importer les éléments supplémentaire requis pour utiliser une fonctionnalité ou méthode précise.

Le premier import :

```
<script type="importmap">
{
  "imports": {
    "three": "https://unpkg.com/three@0.164.1/build/three.module.js",
    "three/addons/": "https://unpkg.com/three@0.164.1/examples/jsm/"
  }
}
</script>
```

Le type=importmap est une fonctionnalité du navigateur qui permet de définir un objet Json qui mappe les noms de modules javascript. Cela permet d'utiliser les imports "nus" (ex: `import {x} from 'module'`) sans outils de construction.(webpack, Vite)

Le deuxième import :

```
<script type="module">
import * as THREE from 'three';
import { OrbitControls } from 'three/addons/controls/OrbitControls.js';
import { STLLoader } from 'three/addons/loaders/STLLoader.js';
import { STLExporter } from 'three/addons/exporters/STLExporter.js';
```

Le type module est nécessaire pour indiquer au navigateur qu'un script doit être traité comme un module. Le navigateur récupère et évalue automatiquement le module importé (et ses importations, le cas échéant), puis exécute le script.

Les fonctions et méthodes

Lors de l'exécution du code javascript, deux fonctions hyper importantes sont exécutées en premier. Le `init()` et le `animate()`, le première comme son nom l'indique initialise le visualiser 3d plusieurs fonctionnalité sont donc mis en route. Le deuxième lui gère l'animation sans lui pas de rafraichissement du visualiser et aucun aperçu.

Pas mal de fonction sont logés dans ces deux méthodes je vous invites à consulter la documentation pour en savoir plus en détail.

La vérification

Comme dans tout opération CRUD il est crucial de faire une vérification que tout les données sont conforme avant de l'envoyer dans la base de donnée. C'est donc la mission de la méthode `checkFormValidity()`.

Création et Update, leur différence

Comme je l'ai dit au début de ce chapitre, nous utilisons quasiment le même principe pour la modification à quelques détails près. Comme par exemple les paramètres d'un job qui sont repris initialement de la base de données.

Une grande différence aussi c'est la possibilité de changer l'état du job dans lesquels il se trouve. Cette partie est pour l'administrateur lorsqu'il gère un dépannage sur une des imprimantes suite à une erreur il peut la remettre en état de marche.

Ainsi, une dernière différence réside aussi dans la gestion du fichier STL en effet, nous chargeons le fichier stl existant actuellement mais il est aussi possible de littéralement changer de pièce.

Conclusion

Cette présente documentation à été rédiger par Gane Gyan, et n'est pas complète en raison du fin de module mais contient le nécessaire pour pouvoir reprendre son développement dans le futur